

Part two — the technical walkthrough

How the system is designed, and what building the prototype taught us about it.

The design in one sentence

We trade speed to merge — and accept that some good changes will be wrongly held back — in exchange for a verdict that the author of a change cannot influence.

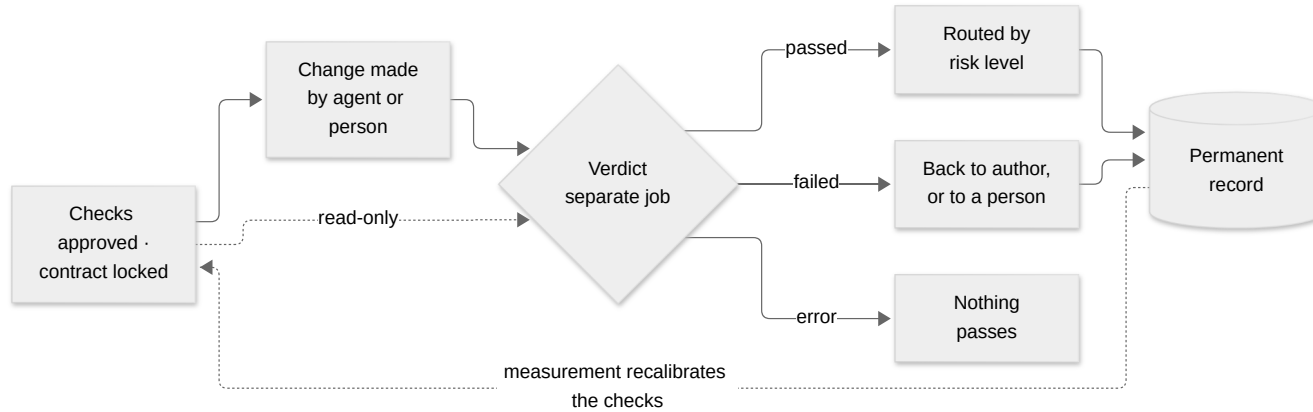
Every quality the design promises is paid for, and we can say with what:

Promise	The rule that delivers it	What it costs us
Integrity	The author cannot alter the checks, the contract or the verdict	Time: the verdict runs as a separate job after the author finishes
Privacy	Code, checks and records never leave the customer's systems	We cannot learn across customers, and support is harder
Reproducibility	Any past verdict can be re-run from its record	Storage, and the discipline of versioning everything

The four objects everything is built from

Object	What it is	The detail that matters
Check	One runnable test of one thing, with a version	A check is either <i>visible</i> to the author or <i>hidden</i> — mounted only where the verdict runs, so the author cannot train against it
Contract	The agreed meaning of "done" for one task: scope, required checks, budget	Lives on a protected branch no author can write to; approved by a person; the pipeline pins its exact fingerprint
Verdict	passed · failed · inconclusive · error · overridden	"Error" is never a pass. When a person overrides the verdict, that override is kept as a label — the raw material for measuring the verdict's own error
Outcome record	One per change, written once, never edited	Who asked, what changed, which checks ran, the verdict, the cost — enough to replay the whole decision years later

How a change flows through it



It attaches as a required check in the team's existing build pipeline — no cooperation needed from any assistant vendor. Assistants can offer early feedback while the work runs; the pipeline's verdict is the one that counts.

How a verdict is computed — and whether it is deterministic

A **verdict is computed, not judged**. Each check runs and reports; the verdict is the worst result, taken in a fixed order, failing closed. It never reads the agent's conversation.

	Ordinary code	Software built on a model
The checks	The team's tests, scope, dependency and secrets screens; the author's new tests re-run on the <i>original</i> code	Evaluation cases for every requirement, some kept hidden from the author
Computed how	Each check runs once	Each case runs several times, within budget; code scores first, a measured model only where code cannot
Deterministic?	Yes — same inputs, same verdict. Flaky checks go to a person, never silently through	No — and it never needed to be. Still fixed in advance, protected, run beyond the author's reach, recorded, error-measured
Outcomes	passed · failed · error	passed · failed · inconclusive · error

The order of the checks, as built and exercised

First match wins; the cheapest decisive checks sit at the top:

1. **The verdict process itself failed** → "error", retried once, then escalated — never a pass
2. **Contract, protected file, existing test or check altered** → failed, in under 0.1 s, before any code runs — the rung that caught the "patched reporter" trick
3. **Budget exceeded** → failed
4. **The build fails** → failed
5. **Any required test fails** — the team's, the author's, or the hidden ones → failed
6. **The change left its agreed scope** → failed
7. **The author's new tests also pass on the original code** → failed — tests that cannot tell old from new prove nothing
8. **A missing dependency, or a secret in the change** → failed
9. Otherwise → passed

Six of our nineteen rejections were decided at step 2, at effectively zero cost.

The hard case: verifying software that is itself built on a model

An AI feature answers differently every time, so a single passing run proves almost nothing. For these, the check is an **evaluation**:

- **The cases are written in advance and versioned**, covering every requirement — and some are hidden from the author, because an author tuning against the full set learns the set, not the task.
- **Each case runs several times**, within the contract's budget.
- **Scoring prefers code over judgment**: exact matches, schemas, tolerances. A model scores only what code cannot — pinned to a version, from a different vendor than the author where possible, and its agreement with human labels is measured and stated.
- **The decision respects uncertainty**: "passed" only when the whole plausible range clears the bar; "failed" only when all of it misses; otherwise **"inconclusive"** — more samples while budget remains, then a person decides.

Built and shown end to end: the original reminder prompt failed 0 of 40 samples; an agent's rewrite passed 40 of 40, every hidden case held — and both of the evaluation's own error sources appeared on the way, were caught from the records, and were fixed.

Who decides how much checking a change gets: people, by risk

Risk level	Who sets it	What happens on "passed"
Low	Rules written by a person — well-tested, isolated areas of the code	Accepted automatically
Normal — the default		A reviewer sees it, evidence first, code second
High — sign-in, payments, permissions, anything the team names		A person is involved at every step; an agent may propose, never land

Two principles. **People draw the lines, not models** — published practitioner guidance is blunt that a model's confidence is a poor guide to risk. And **autonomy is earned with numbers**: a low-risk zone widens only when the

Measuring the measurer

A verdict of unknown reliability is just another opinion. So the product measures its own two error rates, continuously:

- **False passes** — bad changes it let through. Measured with **planted flaws**: changes we deliberately write to be wrong in known ways (scope violations, weakened tests, tampered checks, quietly dropped behaviour) and feed to the gate.
- **False fails** — good changes it wrongly blocked. Measured with known-good changes. A gate that blocks good work does not get tolerated for long; it gets bypassed.

Every rate is reported with its sample size, per kind of task and per model. A model upgrade is treated like a release: the measurement re-runs, because a check that earned its keep on this model may be pure overhead on the next — **that recurrence is the product's subscription logic, not an afterthought.**

What we already know is weak — including what the build itself taught us

We reviewed our own design critically before building, and then the build confirmed and extended the list:

1. **Running the team's tests means executing the author's code** — in the very place the hidden checks live. The prototype's answer: every run happens in its own disposable container with no network and no credentials, and automated tests prove the gate reads neither the answer key nor the agent's transcript. The production design goes further.
2. **The weakest link in a pipeline is its configuration**, which authors can usually edit. The real anchor must be the repository host's required-check mechanism, with the contract on a branch no agent-wielding account can write to. Not yet exercised.
3. **The infrastructure will lie to you occasionally**. During the build, Docker's file sharing twice served stale content — the gate was at risk of judging the wrong version of a change. It now verifies, by fingerprint, inside the container, that it is judging exactly what it was asked to judge. A verifier that judges the wrong files is worse than none, and only our test suite caught it.

What was built, in the order the design demanded

One question: is an executable verdict wrong less often than the agent's own claim, and less often than a rival **model reviewing**? On this task set it now has an answer, and the order in which the instrument was built is why the answer can be defended.

- **Built, in order:** the measurement machinery, proven on a cost-free fake agent first — so nothing would ever be measured on an unproven instrument; then the published 82-line test agent running a real model in containers; then the gate — 350 lines, held by 84 automated tests; then hand-made adversarial changes through it; **only then**, with the answer key strengthened, the 165-run comparison and the rival-vendor reviewer.
- **The alternative** — run the big comparison first and validate the instrument later — would have produced impressive numbers we could not have defended.
- **Deliberately simulated**, by a decision taken up front: the guided check-writing (we wrote contracts by hand) and the protected branch (a directory stood in for it).

Worth saying plainly: we will never ship an agent of our own. The product attaches to the pipelines and assistants a team already has.

Lines we will not cross, in any configuration

1. Never accept a change when the verdict process itself failed.
2. Never let the author of a change write to the checks, the contract, or the verdict.
3. Never let the agent that wrote a change approve it.
4. Never land a high-risk change without a person.
5. Never move a customer's code, checks or records out of their environment.
6. Never use customer data beyond what the customer has explicitly agreed.
7. Never report a pass rate without the verdict's own error rates beside it.
8. Never report a speed or volume number without its quality number.
9. Never run without leaving a record.